
WHITE PAPER · AI OPERATING GOVERNANCE

Confident and Wrong

Challenge protocols for plausible AI output

Marco Policani

Enterprise Portfolio · PMO · AI Operating Governance

policani.net · linkedin.com/in/marcpolicani

July 2026

EXECUTIVE SUMMARY

Plausible output needs a challenge protocol before it changes a decision or an action that matters.

The hardest AI error to catch is coherent, specific, and wrong in a way that survives a quick read. This paper's working claim is that an answer matching what the reader expected gets less scrutiny. The evidence for that is indirect, and the paper says so. The paper proposes replacing "have a human review it" with a five-part challenge protocol. Name the decision the output could change. Try to break the answer, hardest where it matches what was expected. Check its claims against outside sources. Name a reviewer with the time, expertise, and authority to say no, and a route past them. Learn from each failure. No study tests the protocol as a whole.

- 01** Sort output before review by how much its decision matters, so the closest checking goes where a wrong answer would do the most harm.
- 02** Check the claims a decision rests on against sources outside the answer. Asking the model whether it is sure just consults the same witness again.
- 03** Track how often independent checks change a claim. That is the share of checked claims found wrong or unsupported, not the workflow's overall error rate, and no source says how large it should be.

The error that survives a quick read

Fluent AI output can enter work that matters carrying more confidence than evidence, and "have a human review it" often does not catch the problem. This paper takes a position: output that could change a decision or an action that matters should pass a five-part challenge protocol. Name the decision it could change. Try to break the answer. Check its claims outside the answer. Name a reviewer who can say no, with a route past them. Learn from the errors the review catches.

I have reviewed generated analysis that looked excellent: clean structure, plausible figures, and cited sources. One number felt slightly convenient, so I opened the cited report. The report existed. The number did not. The model had blended two adjacent statistics into one that said what the narrative needed, and nothing in the document

gave that away. The only signal was that it agreed with us a little too well. Catching it took a suspicion and the time to act on it.

That is the governance problem in miniature. The hardest AI error to catch is not nonsense; nonsense filters itself. It is the answer that is coherent, specific, and shaped to what the reader expected, and wrong in a way that survives a quick read. Plausibility makes trusting it easy, and agreement may make it easier still.

This paper's working claim is that an answer that confirms what someone already thinks gets less scrutiny than one that argues back, so the checking lands where it is least needed. No study cited here measures that for AI answers at work. The claim rests on two earlier findings and one step of the paper's own. Psychologists call the general tendency confirmation bias: seeking or interpreting evidence in ways that favor beliefs a person already holds.[1] Researchers at Anthropic, an AI developer, found that in human preference data a response was more likely to be preferred when it matched the user's stated views, and in a small proof-of-concept test, non-expert raters without fact-checking tools sometimes preferred a convincing answer that agreed with a mistaken belief stated in the question over a correct one.[2] Neither finding measured how closely people check an AI answer that agrees with them. That step, from these findings to review at work, is this paper's inference.

Models also lean toward agreement: the same study found it in all five AI assistants tested,[2] and in April 2025 OpenAI rolled back a GPT-4o update it described as "overly flattering or agreeable".[3]

This is a documented failure of the technology. A survey of natural-language generation research describes models that are fluent and coherent yet prone to produce unfaithful or nonsensical content.[4] One study put a number on it in a field with real consequences. Dahl and colleagues asked four models from 2023 specific, verifiable questions about random federal court cases. ChatGPT 4 hallucinated on 58 percent of the questions, meaning its answer was inconsistent with legal fact, and Llama 2 on 88 percent.[5] The models also often failed to correct a user's mistaken legal premise.[5] Those models have since been replaced, and the study covers one field. It shows that the rate can be high where the stakes are real, not what the rate is for today's models.

"Have a human review it" is the stock answer, and as stated it is a hope with a name attached. Review that works has structure. It knows what decision the output feeds. It hunts for what would disprove the answer instead of polishing it. It follows claims back to sources. It is done by someone with the time and the standing to say no. It also leaves a trace the organization can learn from. None of that runs on care alone. A careful reviewer can still give the least scrutiny to the most agreeable answers,

which is why review has to be a control: a step built into the work, with an owner, that can be checked.

This paper is written for AI program owners, risk leaders, workflow owners, and executives who depend on generated analysis. The protocol is its own design, drawn from reviewing generated analysis, and no study tests it as a whole. It covers what happens to an output after the tool produces it. A companion paper, "AI should make people better thinkers, not just faster producers," covers how a person forms their own view before asking AI and how to keep their skills from fading.

Why ordinary review misses this error

Reviewers are trained by human failure, and generated output fails differently. This paper's reading is that a weak analysis from a colleague usually shows in how it reads: gaps in the logic, hedged language, formatting that betrays hurry. Fluent generation removes those signals. The prose is confident, the structure is textbook, the hedges are gone, and none of that tracks with whether the answer is right. A reviewer spends a career learning that polish predicts care. Here that lesson misleads.

Human-factors researchers have studied a related effect for decades. Automation bias is the tendency to over-trust an automated aid and under-check its output. A review of that research finds that it leads people to follow wrong recommendations or miss problems the aid failed to flag, that it appears in experts as well as novices, and that training or instructions do not prevent it.[6]

One experiment shows the cost of a persuasive aid. In an unreviewed preprint, 257 medical students at one hospital made 3,855 diagnostic decisions. When the AI was right, which was 73 percent of the cases, AI explanations raised their accuracy by 6.3 percentage points. When the AI was wrong, which was 27 percent of the cases, explanations lowered it by 4.9 points.[7] Across all cases the explanations still helped, by about three points. The authors attribute the loss to explanations that sound equally convincing whether the AI is right or wrong, and report that students kept trusting recommendations that were wrong.[7] The second half is the one that matters for review: this paper treats a convincing explanation as no evidence that the answer is right.

This paper expects the error to compound as people work with it. Someone gets a nearly right answer and refines it, and each round makes the document more persuasive without touching the premise underneath. The person who knows the document best is then the least able to see the problem, because they built their understanding on the same premise the document did.

Volume makes it worse. Generating text is cheap, so output multiplies. Review capacity is human, so it does not. Unless something concentrates scrutiny where the stakes are, the ratio of checking to producing is likely to slip, and the gap may show only when an error does real damage. Mandating review of everything is one response; this paper expects it to collapse under its own weight, and the organization is back to hope.

The five-part challenge protocol

The protocol is five questions, sized to what is at stake. It is deliberately not a checklist for every generated sentence; low-stakes help should stay fast. It is the standard path for output that could change a decision or an action that matters.

EXHIBIT 1

Five challenges stand between fluent output and a decision

- 1 **Name the decision it could change**
How hard you look should follow what the output may change, not how polished it looks.
- 2 **Try to break the answer**
Test the favored conclusion instead of asking the model to make it sound stronger.
- 3 **Check the claims outside the answer**
Follow the important claims to sources outside the generated answer.
- 4 **Name a reviewer, and a route past them**
Review works only when the reviewer has time, expertise, authority, and somewhere to send a doubt.
- 5 **Learn from the errors you catch**
A rejected answer is evidence about prompts, data, interfaces, review load, and permitted use.

The parts mostly depend on each other in order. The first decides whether the rest apply and how hard. The second feeds the third: what the attempt to break the answer turns up is the first list of claims to check, though the third also covers the claims the decision rests on, whether or not the second found a problem. The fifth learns from what the others recorded, so it comes last. The one exception is the fourth. For tier-three output, the reviewer has to be named right after the first part, because that person runs the second and third. In tier two the requester runs them.

Name the decision it could change

Start by naming what the output might change: which decision, affecting whom, how reversible, at what cost if it is wrong. Until someone states that, the wrong things set how hard people look. Document length. Deadline pressure. How polished the text is. None of those is the stake.

This prevents a specific failure: reviewing a document without knowing it feeds something that matters. Consider a reviewer who copy-edits a generated summary as prose. If its headline figure later anchors an investment recommendation, nobody decided that figure deserved investment-grade checking, because at review time nobody knew where the summary was headed, and nobody wrote it down. The stakes were set by drift.

The practice is a short use declaration above the output, written by the requester or workflow owner before review starts: which decision it affects, who is affected, how reversible it is, what an error costs, what the requester expected the answer to say, and how the output will be used, whether for brainstorming, drafting, a recommendation, or the basis for an action. Writing down the expectation matters for the next part. It should take about a minute, which is a planning estimate, not a measurement. It sorts the output into the right review tier and leaves the record that makes the rest of the protocol checkable later. Low-stakes uses stop here, by design: the first decision the protocol makes is whether the protocol applies.

A workable default has three tiers. Tier one is ideas and internal drafting: no protocol, full speed. Tier two is content feeding a recommendation: look for evidence against it, and spot-check the claims it rests on. Tier three is content that will justify an action affecting money, customers, staff, or the public record: the full protocol, a named reviewer, a verification log. When no one can say what decision an output feeds, as with a general-purpose chat tool, treat it as tier two until someone names the use. The exact tiers can be adjusted. What matters is that the sorting happens before review, on the stakes someone declared, rather than after an incident, on regret.

Try to break the answer

The instinct with a promising answer is to make it stronger: ask for more support, more polish, more confidence. The second move runs the other way. Try to break it. What observation would make this conclusion false? What other explanation fits the same facts? Who would dispute it, and what is their best argument? Where does the answer hold only inside one framing? Break hardest where the answer matches the expectation written in the use declaration, because that is where this paper's working claim says scrutiny is thinnest. If the answer survives only inside one framing, narrow the conclusion or withhold it.

This prevents the echo loop: refining an answer round after round until it is more persuasive but no better supported. This paper's inference is that a model's lean toward agreement [2][3] can speed the loop up, since a model that leans that way can amplify whatever framing it is given and someone who prompts from a preferred conclusion may get better-dressed versions of it. Neither source tested repeated refinement.

Breaking an answer is cheap, and it often uses the same tool. Ask the model for the strongest case against its own answer. Ask what circumstances would make it fail. Ask what data would tell it apart from the next-best explanation. Treat what comes back as leads, not findings: the next part checks them outside the answer. Then check whether any of it was already on hand and simply unexamined. A conclusion earns confidence by surviving a serious attempt to break it. Polishing proves nothing, and the reviewer should be able to say which of the two has happened.

Consider a generated competitive analysis that concludes a rival's pricing change signals retreat from the mid-market. Three questions. What would retreat predict that we could see, such as hiring freezes, partners leaving, or features dropped, and do we see it? What else explains the pricing change: a bundling strategy, a fiscal-year discount cycle? Who on the sales team has evidence against the retreat reading? Suppose the third question turned up two account executives who had lost mid-market deals to that rival the same month. The retreat thesis would not survive to the strategy meeting. That is what the third question is for.

Check the claims outside the answer

Verification means following the important claims to ground outside the answer. Open the cited report. Re-run the calculation. Read the policy clause itself. Confirm the quotation. Inside the answer, everything is testimony from the same witness. Checking the answer against itself, by asking the model whether it is sure or asking the same session for sources, is corroboration theater: it looks like a second opinion and is the same witness repeating itself. If no outside source supports a claim the decision turns on, take the claim out of the decision or lean on it less.

This prevents the blended-statistic story and its whole family. Citations to real sources that say something adjacent. Quotations lightly paraphrased into a new meaning. Calculations right in method and wrong in one input. Policy readings that were true two revisions ago. Each one looks like evidence. Each one survives a reviewer who checks that sources are cited without checking what the sources say.

This stays affordable because the target is narrow. Check the load-bearing claims, the ones the decision actually turns on, not every claim. For each, the record is small: source, author, date, the passage or data that supports the claim, and who checked

it. That record is what lets a later reader tell what was verified from what was never examined. It is the difference between a document that carries evidence and one that carries formatting.

Two mechanics raise the yield. First, require generated citations in a form someone can open: a working link or a document reference, not a bare title. A citation nobody can open counts as an unsupported claim. Second, keep the verification log beside the document rather than in a separate application. A check that costs a context switch is likely to stop happening in busy weeks.

Name a reviewer, and a route past them

Review is a control only when a named person takes responsibility for what happens to the output. That person needs relevant expertise, protected time, and somewhere defined to send what exceeds their authority. Missing any of those, review is a signature loop: present, recorded, and useless.

The route upward needs one more property: it has to be safe to use. If sending a contested case up reads as incompetence, reviewers will settle their doubts quietly on their own, and the route becomes decoration. Thank the reviewer, decide the case promptly, and the route stays alive. Until a doubt is resolved, the output does not feed the decision.

This prevents approval in name only, which can arrive through three doors. Volume: suppose a reviewer clears dozens of items a day, and clicking through becomes the job. Wrong expertise: the reviewer can judge the prose but not the claim. No authority: the reviewer has doubts and no standing to stop what happens next, so the doubt dies in a comment. Microsoft's red-teaming report makes the general point that finding failure takes people with the expertise and judgment to look for it.[8] This paper's inference is that the same holds for the review a workflow runs every day.

The fixes are ordinary management, applied on purpose. Match reviewer tiers to the tiers the use declaration produced. Cap review volume before quality collapses, and treat that cap as a limit on how much gets generated. How to count that review load, and why it can become the first limit on AI agents, is the subject of "The Limit on AI Agents May Be the People Who Supervise Them." Separate duties where the requester's interest conflicts with honest review. Define a named route for contested or novel cases. Record the decision and the reason for it, because "approved, with these caveats" is knowledge the organization needs at the next part of the protocol.

Reviewer load is a design input, not a character flaw. Automation researchers find complacency when other work competes with the automated task for attention.[6] This paper infers that a reviewer with many items competing for attention checks each one less, and that a long run of routine approvals trains the reviewer to approve

the next. Rotate reviewers. Plant known-bad items to keep detection sharp. These are inspection practices the paper borrows; no source here tests them for AI review.

Learn from the errors you catch

A failure someone catches is operating evidence. The blended statistic, the misread policy clause, the calculation with the stale input: each one tells you where the workflow's defenses are thin, which prompts and data invite trouble, and what a given review load is missing. Kept private, the catch protects one document. Fed back, it improves the control for everyone.

This prevents organizational amnesia. Corrections happen inside individual sessions: someone notices, fixes it, moves on. The same failure can then repeat across other people and later months, because nothing connects the catches, and the organization pays for the same lesson again in near misses.

The practice is light capture with real routing. Keep a failure log: the category, the workflow, where it was caught, and how much was at stake. Review it on a schedule, looking for repeats. Give someone authority to change something when the same failure keeps showing up: prompts, source data, interface defaults, permitted uses, review tiers, or the test set used to evaluate the tool. Near misses count double, because a failure caught by luck is a failure the control missed. The test of whether this part works is specific: repeating failure types should stop repeating. When failures outrun the organization's ability to catch and correct them, reduce or suspend the permitted use.

A starter set of categories makes the log useful right away. Fabricated support: sources or figures that do not exist as cited. Blended fact: real pieces combined into a false composite, like the blended statistic above. Stale truth: accurate when the model learned or retrieved it, wrong now. Wrong frame: correct information applied to the wrong jurisdiction, segment, or policy version. Induced agreement: the answer bent toward what the prompt visibly wanted. These five are a starting set, and the log should add its own. Where entries cluster tells the program where to spend. Fabrication points at verification discipline, stale truth at refreshing the data, induced agreement at prompt and interface design.

The protocol in real use

Each part is easy to escape alone, so the five cover for each other. A use declaration with no attempt to break the answer sorts the output but never tests it. Breaking the answer without checking it outside breaks framings but not fabrications. Outside checks with no real reviewer produce verified inputs to a decision nobody owns. All

four without learning leave the organization exactly as exposed next quarter. The first part is also what keeps the cost down: this paper expects most output to stop at the declaration, so scrutiny is spent only where the stakes concentrate.

EXHIBIT 2

What each challenge decides

Name the decision it could change

Apply the protocol when depending on the output could do real damage; keep low-risk help light.

Try to break the answer

Narrow or withhold the conclusion when it survives only inside one framing.

Check the claims outside the answer

Remove the claim or lean on it less when no outside evidence can be found.

Name a reviewer, and a route past them

Send a high-stakes doubt to a qualified owner instead of counting any human touch as review.

Learn from the errors you catch

Reduce or suspend permitted use when failures outrun the organization's ability to catch and correct them.

Two realities limit the protocol. First, under deadline pressure the temptation is to skip to the signature. The fix is placement: put the questions in the interface or the work record where people actually depend on the output, not in a policy document nobody opens at 4 p.m. Second, the protocol can be faked: declarations filled in by rote, break-it prompts run and ignored. The fix is sampling: periodic spot-checks that re-verify a handful of decisions, with visible consequences when the record and the reality diverge. Unaudited, the protocol decays into a form.

The protocol is not the same thing as systematic evaluation, and confusing the two wastes both. Evaluation tests the tool, using representative cases, thresholds, and re-tests after a change. It answers whether this workflow should depend on this tool at all. The challenge protocol tests one case: this output, feeding this decision, today. Evaluation results set the protocol's tiers, since a workflow with strong evaluation evidence can verify by sampling rather than checking everything, and the protocol's failure log hands evaluation its most valuable cases. Evaluation cannot see the one convenient number, and case-by-case review cannot see quality drifting slowly across the whole.

Four objections: distrust, better models, speed, and proof

The first objection is that this is distrust of AI. The protocol does not assume every model answer is false, or that a human is automatically right. It adds deliberate checking steps where the stakes warrant it. The safeguards are outside evidence, qualified judgment, and action you can walk back. General-purpose distrust is not a safeguard.

The second objection is that models are improving, so the protocol will age out. Better models may well shrink the error rate, and the Dahl figures above come from models that have been replaced. A lower rate still leaves the damage of the errors that remain, and this paper expects the remaining ones to be harder to spot, because the obvious ones go first. No source here tests that. A control keyed to what is at stake rather than to how often errors occur is the kind that survives better models. The tiers stay, and verification volume can fall once the failure log shows which checks can relax and the checkers pass the planted-item or sampled re-check test.

The third objection is speed. Yes, the protocol slows the last step of work that matters. That is what it is for. Frictionless speed is the wrong benchmark. The likely alternative is the incident, such as the fabricated figure in the board deck or the misread policy in the customer letter, and the blanket review mandate that is likely to follow it and slow everything. The cost of the protocol is a minute per declaration and reviewer time on tier-three items. Both are planning estimates, and the failure log is where the real cost shows. Checking sized to the stakes, done at the moments that matter, is the cheap version of trust.

The fourth objection is that nothing proves it works. That is right. No study cited here tests this protocol, or any one of its five parts, on AI output. The sources show the problem: fluent errors, a lean toward agreement, over-trust in automated aids, and persuasive explanations that mislead when the AI is wrong. The protocol is this paper's design for that problem. The test is in the record. If repeating failure types do not stop repeating, the protocol is not working. If the outside checks in a tier almost never change a claim, test the checkers before loosening anything: plant known-bad items or re-check a sample, and loosen only if the checkers catch them.

Where to put it, and who owns it

Installation is deliberately small:

- Put the five questions where people actually depend on the output: the interface or the work record.

- Decide which decisions need outside checks and which can be sampled.
- Set reviewer capacity before raising how much gets generated.
- Review repeated failures monthly, and change prompts, data, permissions, workflow, and guidance together.

What that looks like depends on the tool, and the details decide whether anyone adopts it. In a document workflow, the use declaration is a template header and the verification log is a table at the end. In a ticketing or CRM flow, the declaration is two required fields and the reviewer's decision is a status. In a chat interface, the break-it pass is a pinned follow-up prompt the team actually uses. None of this needs new software on day one; it needs the questions placed where the work already happens.

Start with one workflow where the stakes are real and the volume is manageable, such as contract analysis, customer-facing recommendations, or financial summaries, and run the protocol there until it is boring. The early weeks may turn up catches nobody expected. That is the baseline being found, not a new problem. Then expand one workflow at a time rather than by mandate, letting each area set its own tiers and sampling.

Ownership splits cleanly. Workflow owners hold the use declarations and the reviewer rosters. The AI program team holds the failure log and the failures that cross workflows. The risk team holds the sampling audit. The protocol should not grow a bureaucracy; three roles and a monthly review carry it.

What the record shows when it works

A working protocol leaves numbers:

- The share of output landing in each review tier.
- Load-bearing claims checked outside the answer, and how often checking changed the claim.
- Reviewer volume against the caps, and decisions recorded with reasons.
- Failures logged by type, where they were caught, and what was at stake, near misses included.
- Repeating failure types retired by a change to the control, and sampled decisions that survived a re-check.

Resist rolling these into a single compliance percentage. One "protocol adherence" score invites exactly the theater the sampling audit exists to catch, and it hides the interesting signals: which kinds of claims keep failing verification, whose decisions keep surviving re-checks, and which workflows keep turning out to matter more than

they declared. These numbers are a diagnostic set, and their disagreements are the findings.

Watch the second measure early. How often an outside check changes a claim is the share of checked claims that turned out wrong or unsupported. Only the load-bearing claims get checked, so it is not the workflow's overall error rate, and no source here says how large it should be. Expect it to differ widely between workflows. If it is near zero in a tier, test the checkers first with planted known-bad items or a sampled re-check, because a near-zero number from checkers who would miss errors is not a safe number. If it is high, the same number says where to look first. Watching it fall as prompts, data, and guidance improve is the protocol's return, made visible.

What changes when review becomes a control

Once challenge is a control, review stops depending on individual virtue and runs as part of the work. Fluency stops working as evidence, because the workflow no longer accepts it as any. The confident and wrong answer will keep arriving; that is the nature of the tools. What changes is what it meets: a reader with a quick eye and a deadline, or a protocol written for exactly this kind of answer.

Notes and sources

[1] Raymond S. Nickerson, "Confirmation Bias: A Ubiquitous Phenomenon in Many Guises," *Review of General Psychology* 2, no. 2 (1998): 175-220.

<https://pages.ucsd.edu/~cmckenzie/nickersonConfirmationBias.pdf>

[2] Mrinank Sharma, Meg Tong, Tomasz Korbak, David Duvenaud, Amanda Askell, Samuel R. Bowman, et al., "Towards Understanding Sycophancy in Language Models," arXiv:2310.13548, published as a conference paper at ICLR 2024.

<https://arxiv.org/abs/2310.13548>

[3] OpenAI, "Sycophancy in GPT-4o: What Happened and What We're Doing About It," April 2025. <https://openai.com/index/sycophancy-in-gpt-4o/>

[4] Ziwei Ji, Nayeon Lee, Rita Frieske, et al., "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, 2023.

<https://arxiv.org/abs/2202.03629>

[5] Matthew Dahl, Varun Magesh, Mirac Suzgun, and Daniel E. Ho, "Large Legal Fictions: Profiling Legal Hallucinations in Large Language Models," *Journal of Legal Analysis* 16(1), 2024, 64–93; open version arXiv:2401.01301.

<https://arxiv.org/abs/2401.01301>

- [6] Raja Parasuraman and Dietrich H. Manzey, "Complacency and Bias in Human Use of Automation: An Attentional Integration," Human Factors 52, no. 3 (2010): 381–410. <https://depositonce.tu-berlin.de/bitstreams/cafd2873-814b-4c59-bab1-add42e249d2/download>
- [7] Manshu Khanna, Ziyi Wang, Lijia Wei, and Lian Xue, "When Medical AI Explanations Help and When They Harm," arXiv:2512.08424, 2025. <https://arxiv.org/abs/2512.08424>
- [8] Blake Bullwinkel et al., "Lessons From Red Teaming 100 Generative AI Products," Microsoft Research, January 2025. <https://www.microsoft.com/en-us/research/publication/lessons-from-red-teaming-100-generative-ai-products/>

About the author

Marco Policani is an enterprise portfolio, PMO, and AI operating-governance leader. He builds portfolio governance systems where AI carries the analytical load and named humans own every decision — an approach documented across case studies, walkthroughs, and working governance guides on his portfolio site.

Portfolio & governance library: policani.net/governance · Full portfolio: policani.net · LinkedIn: linkedin.com/in/marcpolicani

This white paper may be shared freely with attribution. © 2026 Marco Policani.